

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN - ĐHQG TP.HCM
KHOA AN TOÀN THÔNG TIN



TÀI LIỆU BÁO CÁO ĐỒ ÁN BYEBUG
(ONLINE JUDGE)

ĐỒ ÁN MÔN HỌC:
LẬP TRÌNH ỨNG DỤNG WEB - NT208.Q24.ANTT

GIẢNG VIÊN HƯỚNG DẪN:
GV NGHI HOÀNG KHOA

NHÓM THỰC HIỆN:
NHÓM 5

THÀNH VIÊN:
TRẦN NHẬT DUY - 24520403
NGUYỄN CAO XUÂN TRUNG - 24521885
TÔ CÔNG HỮU NHÂN - 24521238
NGUYỄN NGỌC THU AN - 24520063

TP. HỒ CHÍ MINH, 2026

Mục lục

CHƯƠNG 1. Tài liệu thiết kế hệ thống ByeBug	4
1. Tổng quan hệ thống	4
2. Kiến trúc tổng thể	4
3. Thành phần chính	4
4. Luồng tương tác chính	5
4.1. Luồng đăng ký/đăng nhập	5
4.2. Luồng xem và tìm kiếm bài tập	6
4.3. Luồng nộp bài và chấm tự động	6
4.4. Luồng quản trị bài tập	7
5. Phân quyền và bảo mật cơ bản	7
6. Chiến lược mở rộng hệ thống (Scalability Strategy)	7
6.1. Mở rộng tầng API Backend	7
6.2. Mở rộng tầng Judge Engine (Worker Scale)	8
6.3. Mở rộng hạ tầng lưu trữ	8
6.4. Cơ chế dọn dẹp và cô lập tài nguyên cục bộ	9
6.5. Tối ưu hóa truy vấn dữ liệu (Caching & Indexing)	9
7. Thiết kế an toàn môi trường Sandbox (Security Hardening)	9
8. Rủi ro kỹ thuật và hướng xử lý	10
CHƯƠNG 2. Thiết kế cơ sở dữ liệu ByeBug	11
1. Tổng quan hệ thống dữ liệu	11
2. Hệ thống các bảng chi tiết (Tables Metadata)	11
2.1. Bảng: users — Quản lý tài khoản học viên	11
2.2. Bảng: admins — Quản lý tài khoản quản trị hệ thống	12
2.3. Bảng: problems — Thông tin bài tập lập trình	12
2.4. Bảng: tags — Danh mục phân loại thuật toán	13
2.5. Bảng: problem_tags — Quan hệ Nhiều-Nhiều giữa Problem và Tag	13
2.6. Bảng: problem_examples — Dữ liệu ví dụ mẫu hiển thị tại đề bài	13
2.7. Bảng: testcases — Hệ thống dữ liệu kiểm thử ẩn dùng để chấm bài	14
2.8. Bảng: submissions — Nhật ký các lượt nộp bài làm của học viên	14
2.9. Bảng: testcase_results — Chi tiết kết quả thực thi trên từng bộ testcase	15
2.10. Bảng: notification_broadcasts — Kho lưu trữ nội dung thông báo gốc từ ban quản trị	16
2.11. Bảng: notifications — Quản lý trạng thái tiếp nhận thông báo của từng học viên	16
2.12. Bảng: admin_notifications — Quản lý trạng thái tiếp nhận thông báo nội bộ của Admin	16

3. Sơ đồ mối quan hệ dữ liệu (ERD)	17
4. Chiến lược lập chỉ mục tối ưu hiệu năng (Database Indexing Blueprint)	17
4.1. Xác thực & Tìm kiếm tài khoản	17
4.2. Bảng xếp hạng (Leaderboard)	18
4.3. Tìm kiếm Bài tập (Problems Discovery)	18
4.4. Chấm bài & Lịch sử hệ thống (Submissions Management)	18
4.5. Truyền tải Thông báo (Notifications Polling)	19
5. Ghi chú thiết kế hệ thống dữ liệu (Design Notes)	19
CHƯƠNG 3. Đặc tả hệ thống REST API ByeBug	20
1. Quy ước chung	20
2. Phân hệ API dành cho Học viên (Auth & Users APIs)	20
2.1. API: Đăng ký tài khoản (POST /api/users/register)	21
2.2. API: Đăng nhập hệ thống (POST /api/users/login)	21
3. Phân hệ API Quản trị phân quyền (Admin Auth API)	22
4. Phân hệ API Bài tập công khai (User Problem APIs)	22
4.1. API: Lấy danh sách bài tập công khai (GET /api/problems)	22
4.2. API: Xem chi tiết đề bài tập (GET /api/problems/{id})	23
5. Phân hệ API Lướt nộp bài (Submission APIs)	24
5.1. API: Gửi mã nguồn nộp bài (POST /api/submissions)	24
5.2. API: Xem kết quả chấm bài (GET /api/submissions/{submissionId})	24
6. Phân hệ API Quản trị bài tập dành cho Admin (Admin Problem APIs)	25
6.1. API: Khởi tạo bài toán mới (POST /api/admin/problems)	26
7. Phân hệ API Quản lý Người dùng dành cho Admin (Admin User APIs)	26
8. Phân hệ API Quản lý Thông báo (Notification APIs)	27
8.1. Phân hệ dành cho Học viên (/api/notifications)	27
8.2. Phân hệ dành cho Admin (/api/admin/notifications)	27
9. Phân hệ API Thống kê & Dashboard (Home / Overview APIs)	27
10. Định nghĩa hệ thống mã trạng thái (Standard HTTP Status Codes)	28
CHƯƠNG 4. Kiến trúc thành phần và triển khai hệ thống ByeBug	29
1. Môi trường công nghệ nền tảng (Technology Stack)	29
2. Cấu trúc tổ chức mã nguồn (Source Code Directory Tree)	29
3. Phân rã cấu trúc Phân tầng lớp của API Backend	30
3.1. Tầng Controller — Điều hướng API	30
3.2. Tầng Service — Xử lý Nghiệp vụ	31
3.3. Tầng Repository — Truy cập dữ liệu	31
4. Phân rã cấu trúc Phân hệ Frontend (React Application)	32
4.1. Tầng Giao tiếp API (frontend/src/api)	32
4.2. Hệ thống Trang (Main Pages) & Thành phần (Components) nổi bật	32

5. Kiến trúc Chuyên biệt của khối chấm bài Judge Engine	32
5.1. Quy trình vận hành cốt lõi (Core Workflow)	33
5.2. Tổ chức Modules ứng dụng áp dụng Strategy Pattern	33
6. Điều phối Hạ tầng ảo hóa Docker Compose	33
7. Hướng dẫn vận hành khởi chạy tại môi trường nội bộ (Local Deployment Guide) .	34
7.1. Cấu hình biến môi trường hệ thống	34
7.2. Lệnh khởi chạy đồng bộ	34
8. Hệ thống Kiểm thử tự động tích hợp (Integrated Unit Tests)	34
CHƯƠNG 5. Đánh giá hiệu năng hệ thống ByeBug	35
1. Mục tiêu đánh giá và Phạm vi đo lường	35
2. Công cụ và Môi trường đo kiểm (Testbed Setup)	35
2.1. Hệ thống công cụ sử dụng	35
2.2. Cấu hình môi trường thử nghiệm	35
3. Kết quả Kiểm thử tải hệ thống các Phân hệ API (Load Testing Results)	35
3.1. Bảng tổng hợp hiệu năng tầng REST API	35
4. Kết quả Đánh giá Năng lực và Tốc độ Chấm bài (Judge Latency Results)	36
4.1. Bảng đo lường thời gian hoàn tất chấm bài (Turnaround Time)	36
5. Phân tích Kỳ vọng và Định hướng Tối ưu hệ thống (Performance Bottleneck Analysis)	37
6. Kết luận Chương Hiệu năng	38

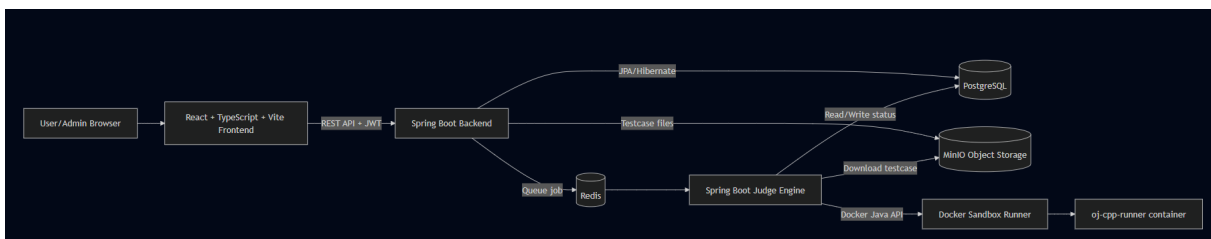
CHƯƠNG 1. Tài liệu thiết kế hệ thống ByeBug

1. Tổng quan hệ thống

ByeBug là một hệ thống Online Judge phục vụ luyện tập lập trình trực tuyến. Người dùng có thể đăng ký, đăng nhập, xem danh sách bài tập, đọc đề bài, viết/nộp mã nguồn, xem kết quả chấm tự động, theo dõi thống kê cá nhân và bảng xếp hạng. Quản trị viên có thể quản lý người dùng, quản lý bài tập, upload testcase, kiểm soát trạng thái hiển thị bài toán, gửi thông báo và theo dõi tổng quan hệ thống.

2. Kiến trúc tổng thể

Hệ thống ByeBug áp dụng mô hình kiến trúc **Event-Driven Architecture (Kiến trúc hướng sự kiện)** kết hợp **Shared Database Pattern**. Sự tách biệt độc lập giữa tầng tiếp nhận nghiệp vụ ngắn hạn (Backend API) và tầng thực thi tính toán nặng dài hạn (Judge Engine) thông qua Message Broker (Redis Queue) giúp hệ thống đạt được trạng thái liên kết lỏng lẻo (Loose Coupling), cô lập hoàn toàn các điểm nghẽn hiệu năng (Performance Bottlenecks).



Hình 1: Kiến trúc tổng thể hệ thống ByeBug

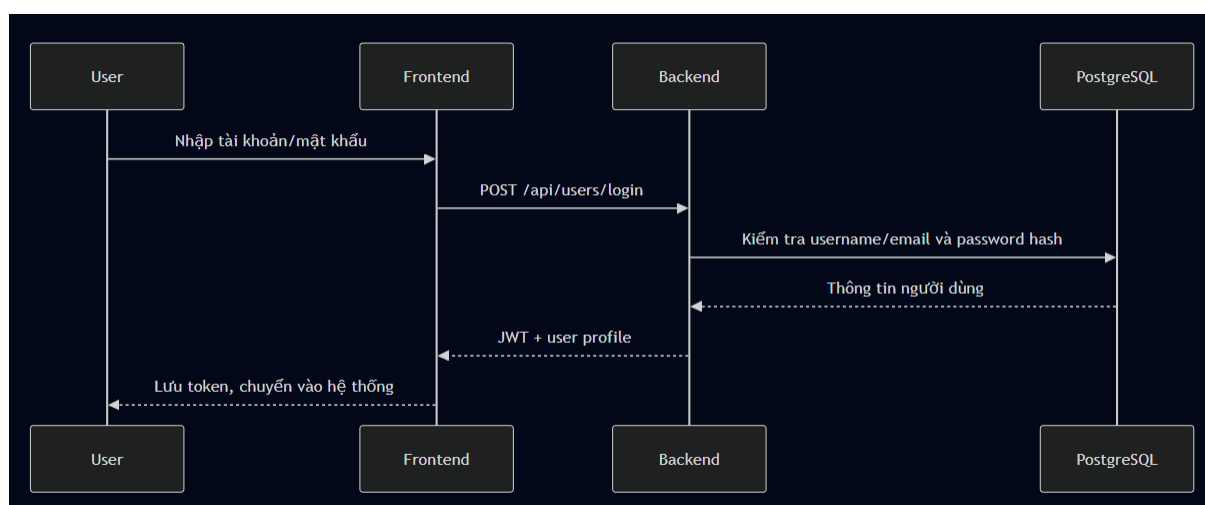
3. Thành phần chính

Thành phần	Công nghệ	Vai trò
Frontend	React, TypeScript, Vite, Axios, Monaco Editor	Giao diện người dùng, trang bài tập, trình soạn code, hồ sơ, thống kê, admin dashboard.
Backend API	Java Spring Boot, Spring Security, JWT, JPA/Hibernate	Xử lý nghiệp vụ chính, xác thực, quản lý bài tập, người dùng, submission, notification.
Database	PostgreSQL	Lưu trữ dữ liệu quan hệ: user, admin, problem, testcase, submission, testcase result, notification.

Thành phần	Công nghệ	Vai trò
Redis	Redis 7	Đóng vai trò Message Broker (Hàng đợi gửi job chấm bài) kết nối bất đồng bộ giữa backend và judge engine.
MinIO	Object Storage (S3 Compatible)	Lưu trữ tệp tin testcase input/output và file nguồn nộp bài nhằm giảm tải dung lượng cho DB.
Judge Engine	Java Spring Boot Worker	Lấy job từ Redis, điều phối tài nguyên chạy code trong Docker, cập nhật trạng thái chấm bài (Verdict).
Sandbox	Docker runner image	Biên dịch và thực thi chương trình của người dùng trong môi trường cô lập, an toàn tuyệt đối.

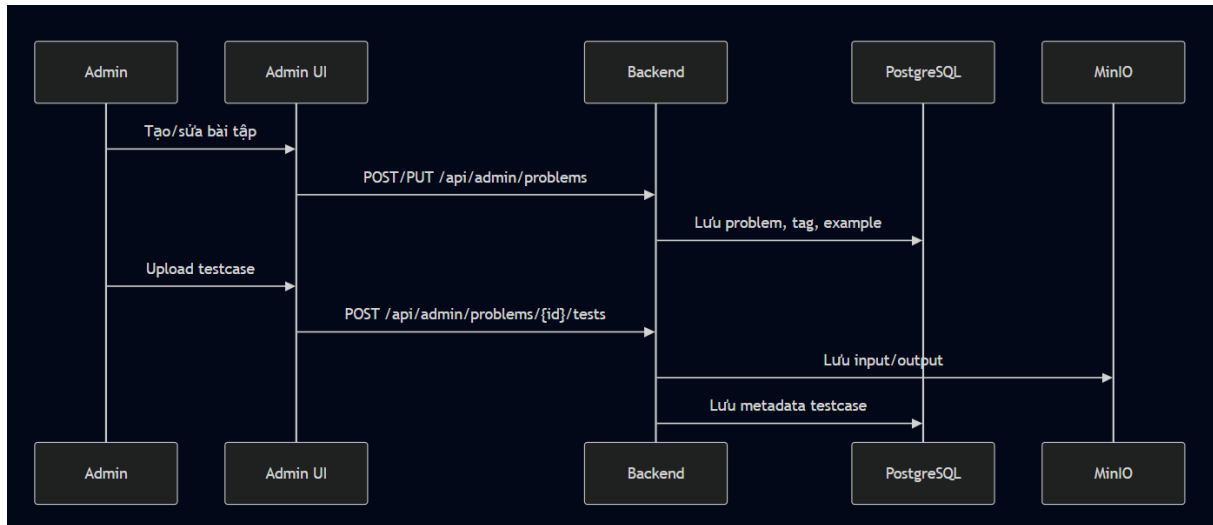
4. Luồng tương tác chính

4.1. Luồng đăng ký/đăng nhập



Hình 2: Luồng đăng ký / đăng nhập

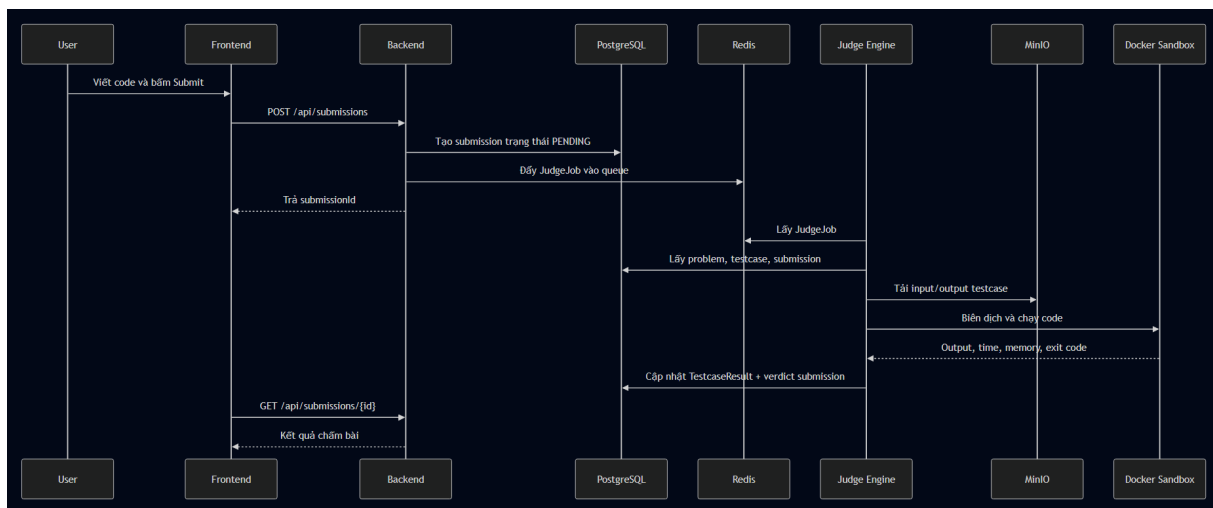
4.2. Luồng xem và tìm kiếm bài tập



Hình 3: Luồng xem và tìm kiếm bài tập

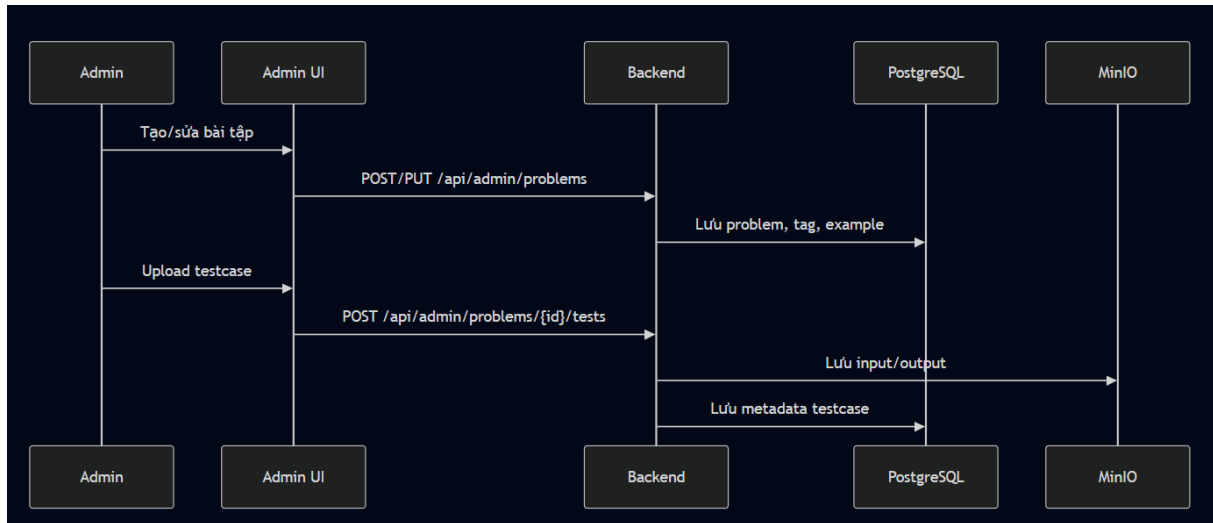
4.3. Luồng nộp bài và chấm tự động

Cơ chế đồng bộ hóa: Hệ thống sử dụng giải pháp Short-Polling từ Frontend với tần suất $t = 2s$ /lần sau khi nhận được `submissionId` từ API Server. Lựa chọn này tối ưu hơn giải pháp kết nối duy trì liên tục (WebSocket) trong môi trường đồ án, giảm thiểu tải kết nối trên API Gateway mà vẫn đảm bảo độ trễ trải nghiệm người dùng (UX Latency) dưới 2000ms.



Hình 4: Luồng submit

4.4. Luồng quản trị bài tập



Hình 5: Luồng đi quản trị bài tập

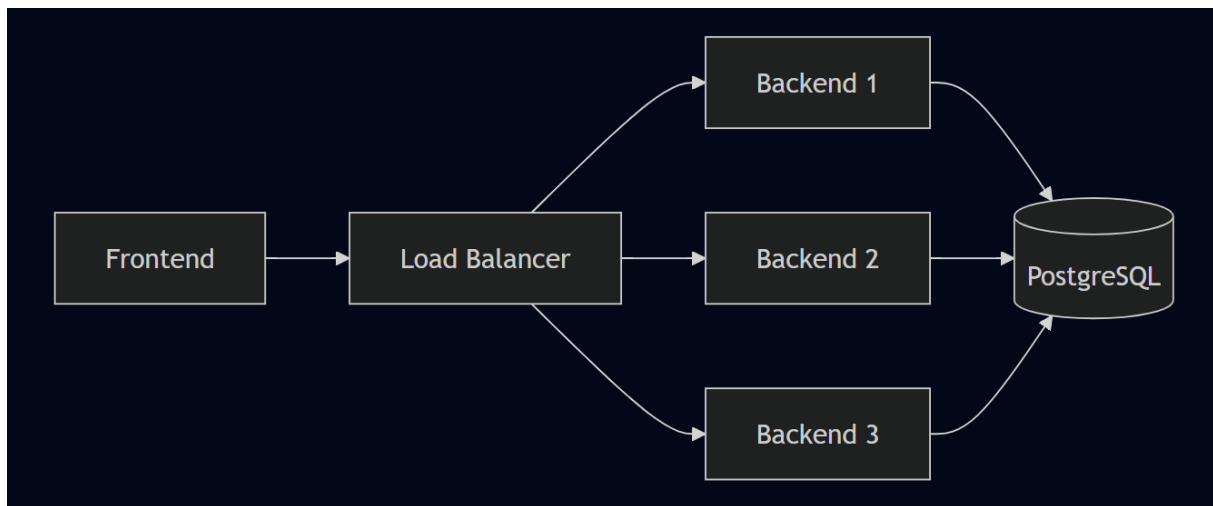
5. Phân quyền và bảo mật cơ bản

- Người dùng thông thường chỉ được quyền tiếp cận các API public/user (đăng ký, đăng nhập, xem bài public, nộp bài, xem thống kê cá nhân và bảng xếp hạng).
- Admin sử dụng nhóm API riêng biệt cấu hình tiền tố `/api/admin/` để thực hiện quản lý bài tập, người dùng, cấu hình thông báo và giám sát hệ thống.
- Backend sử dụng cấu hình Spring Security kết hợp với JWT (JSON Web Token). Frontend có trách nhiệm đính kèm token trong mỗi request tại Header dưới dạng `Authorization: Bearer <token>`.
- Toàn bộ mật khẩu của tài khoản đều được băm bằng thuật toán mã hóa một chiều (Password Hashing) trước khi lưu trữ xuống Database, tuyệt đối không lưu chuỗi văn bản thuần (Plaintext).

6. Chiến lược mở rộng hệ thống (Scalability Strategy)

6.1. Mở rộng tầng API Backend

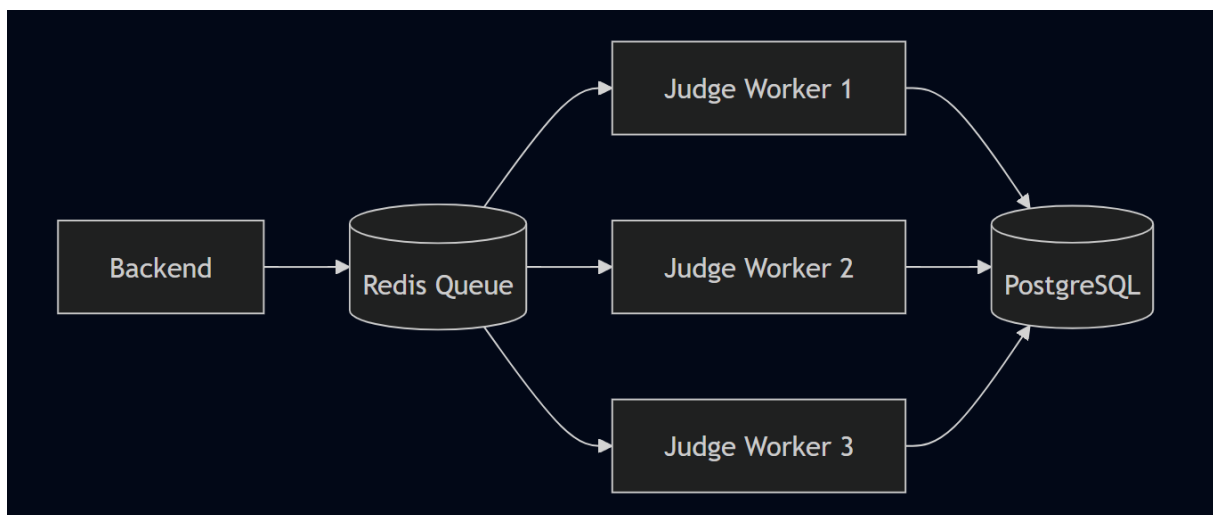
Backend API có thể mở rộng độc lập theo chiều ngang (Horizontal Scaling) bằng cách triển khai đồng thời nhiều Instance Spring Boot đứng phía sau một bộ cân bằng tải (Load Balancer như Nginx hoặc AWS ALB). Do cơ chế xác thực trạng thái người dùng được đóng gói hoàn toàn trong JWT (Stateless Backend), các Node Backend không cần thực hiện đồng bộ hóa bộ nhớ Session vật lý.



Hình 6: Chiến lược mở rộng API Backend

6.2. Mở rộng tầng Judge Engine (Worker Scale)

Judge Engine được thiết kế tách biệt theo dạng Worker để phục vụ mục đích co giãn linh hoạt. Khi tần suất nộp bài tăng cao đột biến (ví dụ trong các kỳ thi), hệ thống có thể tăng số lượng các Judge Worker cùng lắng nghe và tiêu thụ chung một Redis Queue. Mỗi Worker sẽ tự động kéo một tác vụ (Job) riêng biệt, thực thi chấm và cập nhật trực tiếp kết quả về Database mà không gây tắc nghẽn luồng xử lý của API chính.



Hình 7: Chiến lược mở rộng Judge Engine - Worker Scale

6.3. Mở rộng hạ tầng lưu trữ

Tập tin Testcase dữ liệu đầu vào/đầu ra và mã nguồn giải bài tập được lưu trữ tách biệt hoàn toàn tại Object Storage (MinIO) thay vì lưu trữ dạng nhị phân BLOB trong Database giúp kích thước cơ sở dữ liệu luôn gọn nhẹ, dễ dàng thực hiện sao lưu (Backup) dữ liệu định kỳ.

Khi triển khai trên môi trường Production chạy thật, hệ thống có thể chuyển đổi cấu hình sang các dịch vụ đám mây tương thích như AWS S3 hay Google Cloud Storage mà không cần thay đổi cấu trúc mã nguồn.

6.4. Cơ chế dọn dẹp và cô lập tài nguyên cục bộ

Để đảm bảo các Instance Worker khi mở rộng không làm cạn kiệt tài nguyên phần cứng của Host Node:

- **Container Lifecycle:** Mỗi Docker Sandbox Container chỉ phục vụ biên dịch hoặc thực thi duy nhất cho một tác vụ nộp bài, sau khi hoàn thành sẽ lập tức bị xóa bỏ hoàn toàn (`docker rm -f`) để giải phóng tài nguyên bộ nhớ RAM và CPU.
- **Local Cache Expiry:** Toàn bộ file dữ liệu testcase sau khi tải từ MinIO về bộ nhớ đệm tạm thời (Temporary Directory) của Judge Worker sẽ được thiết lập vòng đời tự động xóa sau khi có Verdict, tránh rò rỉ dung lượng đĩa cứng (Disk Exhaustion).

6.5. Tối ưu hóa truy vấn dữ liệu (Caching & Indexing)

- Sử dụng Redis làm tầng Cache đệm lưu trữ dữ liệu cho các chức năng có tần suất truy cập cao nhưng ít biến động liên tục như: Danh sách bài tập tiêu biểu, bảng xếp hạng tổng (Leaderboard).
- Thiết lập phân trang (Pagination) bắt buộc cho toàn bộ các API trả về danh sách lớn (problems, submissions, notifications).
- Khởi tạo chỉ mục (Database Indexing) cho các trường dữ liệu thường xuyên nằm trong điều kiện lọc hoặc sắp xếp: `username, email, difficulty, is_public, submitted_at, total_score, created_at`.

7. Thiết kế an toàn môi trường Sandbox (Security Hardening)

Để bảo vệ máy chủ Host khỏi các cuộc tấn công mã độc hoặc các đoạn code phá hoại hệ thống do người dùng vô tình hoặc cố ý nộp lên, Docker Sandbox Runner được thiết lập cấu hình nghiêm ngặt:

- **Hạn chế đặc quyền (Non-root User):** Tiến trình thực thi mã nguồn của người dùng bên trong Container được chỉ định chạy dưới quyền của User không có đặc quyền hệ thống (`nobody`), loại bỏ nguy cơ chiếm quyền điều khiển tài nguyên vật lý.
- **Cô lập mạng hoàn toàn (Network Isolation):** Khởi chạy Sandbox Container với tham số mạng `--network none`. Đoạn mã của người dùng hoàn toàn bị cắt đứt kết nối Internet, không thể gửi yêu cầu HTTP ra ngoài, không thể tải mã độc hoặc thực hiện các hành vi tấn công DDoS ngược lại mạng nội bộ.
- **Giới hạn tiến trình (Fork Bomb Protection):** Cấu hình chặt chẽ giới hạn số lượng tiến trình con được phép sinh ra trong container thông qua cờ `--pids-limit 64`, ngăn

chặn triệt để các cuộc tấn công từ chối dịch vụ dạng Fork Bomb làm treo hệ thống máy chủ.

8. Rủi ro kỹ thuật và hướng xử lý

Rủi ro	Ảnh hưởng	Hướng xử lý
Mã nguồn người dùng chạy vô hạn	Gây nghẽn luồng xử lý của Judge Worker, treo tài nguyên hệ thống.	Cấu hình giới hạn thời gian chạy tối đa (Time Limit), áp dụng cơ chế tự động ngắt tiến trình (Timeout Process) hoặc cưỡng chế tắt container từ phía Worker.
Mã nguồn tiêu tốn quá nhiều RAM	Gây sập hoặc làm chậm Host Machine (Lỗi Out Of Memory).	Giới hạn dung lượng bộ nhớ tối đa được phép phân bổ cho Container thông qua cấu hình giới hạn RAM của Docker (<code>--memory</code>).
Hàng đợi nộp bài tăng nhanh	Thời gian chờ đợi kết quả chấm bài của người dùng bị kéo dài (UX kém).	Thiết lập hệ thống giám sát độ dài hàng đợi (Queue Length Monitoring), kích hoạt cơ chế tự động nhân bản thêm Judge Worker (Auto-scaling) khi hàng đợi vượt ngưỡng cảnh báo.
Database rơi vào trạng thái quá tải	Độ trễ của toàn bộ hệ thống API tăng mạnh, có thể gây sập kết nối DB.	Tối ưu hóa Index, áp dụng bộ đệm Cache Redis, phân trang nghiêm ngặt, thực hiện tách biệt cơ sở dữ liệu đọc/ghi (Read/Write Replicas) nếu cần thiết.
Rò rỉ dữ liệu Testcase	Làm mất tính công bằng và minh bạch của các cuộc thi trực tuyến.	Cấu hình MinIO Bucket ở chế độ riêng tư (Private Bucket), chỉ cho phép Backend API và Judge Engine truy cập thông qua tài khoản định danh nội bộ có thời hạn Presigned URL ngắn.

CHƯƠNG 2. Thiết kế cơ sở dữ liệu ByeBug

1. Tổng quan hệ thống dữ liệu

Hệ thống ByeBug lựa chọn cơ sở dữ liệu quan hệ PostgreSQL làm nền tảng lưu trữ chính, tương tác thông qua tầng Spring Data JPA/Hibernate tại Backend Server. Dữ liệu tệp tin có dung lượng lớn và tần suất thay đổi thấp (như các tệp tin dữ liệu đầu vào `.in` và đầu ra `.out` của các bộ Testcase) được lưu trữ độc lập ngoài cơ sở dữ liệu thông qua hệ thống MinIO Object Storage; cơ sở dữ liệu PostgreSQL chỉ chịu trách nhiệm quản lý thông tin định danh (Metadata) và đường dẫn tệp tin (Object Key).

2. Hệ thống các bảng chi tiết (Tables Metadata)

2.1. Bảng: `users` — Quản lý tài khoản học viên

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
<code>user_id</code>	BIGSERIAL	PRIMARY KEY	Khóa chính tự tăng của người dùng.
<code>username</code>	VARCHAR(50)	UNIQUE, NOT NULL	Tên đăng nhập hệ thống.
<code>full_name</code>	VARCHAR(100)	NOT NULL	Họ và tên hiển thị trên giao diện.
<code>email</code>	VARCHAR(100)	UNIQUE, NOT NULL	Địa chỉ email người dùng.
<code>password_hash</code>	VARCHAR(255)	NOT NULL	Mật khẩu đã mã hóa một chiều qua BCrypt.
<code>avatar_url</code>	TEXT	NULL	Đường dẫn ảnh đại diện (Lưu trên Object Storage).
<code>gender</code>	VARCHAR(10)	NULL	Giới tính của người dùng.
<code>role</code>	VARCHAR(20)	DEFAULT 'USER'	Vai trò hệ thống, mặc định là USER.
<code>total_score</code>	INTEGER	DEFAULT 0	Tổng điểm tích lũy dùng để xếp hạng Leaderboard.
<code>is_active</code>	BOOLEAN	DEFAULT TRUE	Trạng thái kích hoạt của tài khoản.
<code>created_at</code>	TIMESTAMP	DEFAULT NOW()	Thời điểm khởi tạo tài khoản.
<code>last_login_at</code>	TIMESTAMP	NULL	Thời điểm thực hiện lần đăng nhập gần nhất.
<code>deleted_at</code>	TIMESTAMP	NULL	Đánh dấu thời gian xóa tài khoản (Soft Delete).

2.2. Bảng: `admins` — Quản lý tài khoản quản trị hệ thống

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
<code>admin_id</code>	<code>BIGSERIAL</code>	<code>PRIMARY KEY</code>	Khóa chính hệ thống admin.
<code>username</code>	<code>VARCHAR(50)</code>	<code>UNIQUE, NOT NULL</code>	Tên đăng nhập của admin.
<code>full_name</code>	<code>VARCHAR(100)</code>	<code>NOT NULL</code>	Họ tên hiển thị của admin.
<code>email</code>	<code>VARCHAR(100)</code>	<code>UNIQUE, NOT NULL</code>	Email liên hệ của admin.
<code>password_hash</code>	<code>VARCHAR(255)</code>	<code>NOT NULL</code>	Mật khẩu đã mã hóa của admin.
<code>avatar_url</code>	<code>TEXT</code>	<code>NULL</code>	Đường dẫn ảnh đại diện của quản trị viên.
<code>is_active</code>	<code>BOOLEAN</code>	<code>DEFAULT TRUE</code>	Trạng thái hoạt động của tài khoản admin.
<code>created_at</code>	<code>TIMESTAMP</code>	<code>DEFAULT NOW()</code>	Thời điểm khởi tạo tài khoản quản trị.

2.3. Bảng: `problems` — Thông tin bài tập lập trình

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
<code>problem_id</code>	<code>BIGSERIAL</code>	<code>PRIMARY KEY</code>	Khóa chính bài toán.
<code>title</code>	<code>VARCHAR(255)</code>	<code>NOT NULL</code>	Tiêu đề của bài tập.
<code>description</code>	<code>TEXT</code>	<code>NOT NULL</code>	Nội dung chi tiết đề bài (Định dạng Markdown).
<code>constraints</code>	<code>TEXT</code>	<code>NULL</code>	Các ràng buộc đặc biệt của bài toán.
<code>difficulty</code>	<code>VARCHAR(20)</code>	<code>NOT NULL</code>	Độ khó bài toán: <code>EASY</code> , <code>MEDIUM</code> , <code>HARD</code> .
<code>time_limit_ms</code>	<code>INTEGER</code>	<code>NOT NULL</code>	Giới hạn thời gian chạy tối đa (mili-giây).
<code>memory_limit_kb</code>	<code>INTEGER</code>	<code>NOT NULL</code>	Giới hạn bộ nhớ RAM tối đa cấp phát (KB).
<code>allow_file_submit</code>	<code>BOOLEAN</code>	<code>DEFAULT FALSE</code>	Cho phép người dùng nộp bài bằng cách tải tệp tin.
<code>is_public</code>	<code>BOOLEAN</code>	<code>DEFAULT FALSE</code>	Trạng thái hiển thị công khai trên hệ thống.

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
creator_id	BIGINT	FOREIGN KEY	Liên kết đến admins.admin_id (Người biên soạn).
created_at	TIMESTAMP	DEFAULT NOW()	Thời điểm khởi tạo bài toán.

2.4. Bảng: tags — Danh mục phân loại thuật toán

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
tag_id	SERIAL	PRIMARY KEY	Khóa chính danh mục tag.
tag_name	VARCHAR(50)	UNIQUE, NOT NULL	Tên thẻ phân loại (Ví dụ: Array, Math, DP, Graph).

2.5. Bảng: problem_tags — Quan hệ Nhiều-Nhiều giữa Problem và Tag

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
problem_id	BIGINT	FOREIGN KEY, PK	Liên kết đến mã bài toán problems.problem_id.
tag_id	INTEGER	FOREIGN KEY, PK	Liên kết đến mã danh mục tags.tag_id.

2.6. Bảng: problem_examples — Dữ liệu ví dụ mẫu hiển thị tại đề bài

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
example_id	BIGSERIAL	PRIMARY KEY	Khóa chính dữ liệu ví dụ.
problem_id	BIGINT	FOREIGN KEY	Liên kết đến bài toán chứa ví dụ mẫu này.
input	TEXT	NOT NULL	Dữ liệu đầu vào mẫu hiển thị cho học viên.
output	TEXT	NOT NULL	Dữ liệu đầu ra mẫu kỳ vọng tương ứng.
explanation	TEXT	NULL	Đoạn văn bản giải thích luồng thuật toán của ví dụ.
display_order	INTEGER	DEFAULT 1	Thứ tự sắp xếp hiển thị trên giao diện đề bài.

2.7. Bảng: `testcases` — Hệ thống dữ liệu kiểm thử ẩn dùng để chấm bài

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
<code>testcase_id</code>	BIGSERIAL	PRIMARY KEY	Khóa chính của bộ testcase.
<code>problem_id</code>	BIGINT	FOREIGN KEY	Bài toán sở hữu testcase, kết nối với <code>problems</code> .
<code>input_path</code>	TEXT	NOT NULL	Mã định danh Object Key lưu tệp tin đầu vào tại MinIO.
<code>output_path</code>	TEXT	NOT NULL	Mã định danh Object Key lưu tệp tin đầu ra tại MinIO.
<code>is_visible</code>	BOOLEAN	DEFAULT FALSE	Xác định hệ thống có cho phép hiển thị bộ test này không.
<code>display_order</code>	INTEGER	DEFAULT 1	Thứ tự tuần tự khi Judge Worker gọi chạy test.
<code>score_weight</code>	INTEGER	DEFAULT 10	Trọng số điểm đạt được khi vượt qua testcase này.

2.8. Bảng: `submissions` — Nhật ký các lượt nộp bài làm của học viên

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
<code>submission_id</code>	BIGSERIAL	PRIMARY KEY	Khóa chính của lượt nộp bài.
<code>problem_id</code>	BIGINT	FOREIGN KEY	Bài toán được giải, kết nối với <code>problems</code> .
<code>user_id</code>	BIGINT	FOREIGN KEY	Người thực hiện nộp bài, kết nối với <code>users</code> .
<code>language</code>	VARCHAR(20)	NOT NULL	Ngôn ngữ sử dụng lập trình (cpp, java, python).
<code>source_code</code>	TEXT	NULL	Mã nguồn giải bài (nếu gõ trực tiếp trên Editor).
<code>file_url</code>	TEXT	NULL	Đường dẫn file mã nguồn nếu chọn phương thức nộp file.

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
verdict	VARCHAR(30)	DEFAULT 'PENDING'	Trạng thái chấm: PENDING, AC, WA, TLE, MLE, CE...
compile_error	TEXT	NULL	Nội dung chi tiết lỗi log trả về từ trình biên dịch.
judge_message	TEXT	NULL	Thông báo bổ sung từ hệ thống Judge Engine.
score	INTEGER	DEFAULT 0	Tổng điểm số thu về của lượt nộp bài này.
total_time_ms	INTEGER	DEFAULT 0	Thời gian thực thi lâu nhất trong các testcase (ms).
max_memory_kb	INTEGER	DEFAULT 0	Dung lượng bộ nhớ lớn nhất ghi nhận được (KB).
submitted_at	TIMESTAMP	DEFAULT NOW()	Thời điểm hệ thống ghi nhận lượt nộp bài.

2.9. Bảng: `testcase_results` — Chi tiết kết quả thực thi trên từng bộ testcase

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGSERIAL	PRIMARY KEY	Khóa chính kết quả kiểm thử.
submission_id	BIGINT	FOREIGN KEY	Thuộc lượt nộp bài nào, kết nối với submissions.
testcase_id	BIGINT	FOREIGN KEY	Liên kết đối chiếu sang danh mục bộ test testcases.
verdict	VARCHAR(30)	NOT NULL	Kết quả đánh giá trên bộ test này (AC, WA, TLE...).
user_output	TEXT	NULL	Dữ liệu đầu ra do mã nguồn của người dùng sinh ra.
time_ms	INTEGER	NOT NULL	Thời gian thực thi chi tiết trên cấu hình testcase (ms).
memory_kb	INTEGER	NOT NULL	Bộ nhớ tiêu tốn chi tiết trên cấu hình testcase (KB).

2.10. Bảng: `notification_broadcasts` — Kho lưu trữ nội dung thông báo gốc từ ban quản trị

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
<code>broadcast_id</code>	BIGSERIAL	PRIMARY KEY	Khóa chính thông báo gốc.
<code>title</code>	VARCHAR(255)	NOT NULL	Tiêu đề hiển thị của thông báo.
<code>content</code>	TEXT	NOT NULL	Nội dung chi tiết của thông báo gửi đi.
<code>audience_type</code>	VARCHAR(30)	NOT NULL	Phân loại nhóm đối tượng nhận thông báo.
<code>priority</code>	VARCHAR(20)	DEFAULT 'NORMAL'	Mức độ khẩn cấp của thông báo (LOW, NORMAL, HIGH).
<code>created_by_admin_id</code>	BIGINT	FOREIGN KEY	Định danh admin tạo thông báo, kết nối với <code>admins</code> .
<code>scheduled_at</code>	TIMESTAMP	NULL	Thiết lập lịch phát thông báo tự động trong tương lai.
<code>created_at</code>	TIMESTAMP	DEFAULT NOW()	Thời gian biên soạn nội dung thông báo.

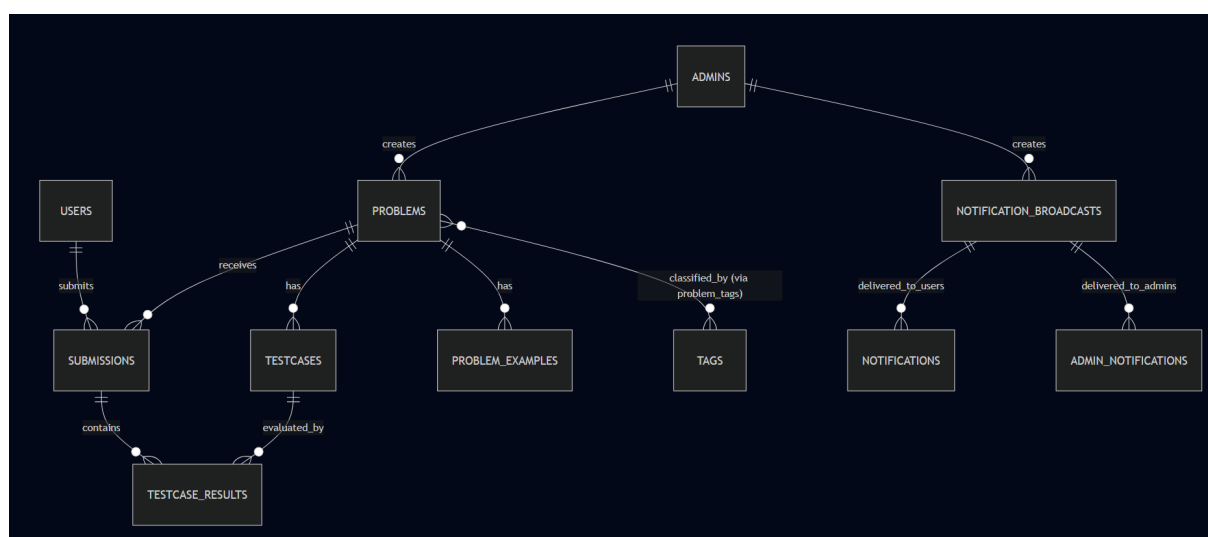
2.11. Bảng: `notifications` — Quản lý trạng thái tiếp nhận thông báo của từng học viên

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
<code>notification_id</code>	BIGSERIAL	PRIMARY KEY	Khóa chính bản ghi nhận.
<code>broadcast_id</code>	BIGINT	FOREIGN KEY	Liên kết sang thông báo gốc <code>notification_broadcasts</code> .
<code>user_id</code>	BIGINT	FOREIGN KEY	Người nhận thông báo, kết nối với bảng <code>users</code> .
<code>is_read</code>	BOOLEAN	DEFAULT FALSE	Trạng thái người dùng đã đọc thông báo hay chưa.
<code>created_at</code>	TIMESTAMP	DEFAULT NOW()	Thời điểm gửi thông báo đến hộp thư người dùng.

2.12. Bảng: `admin_notifications` — Quản lý trạng thái tiếp nhận thông báo nội bộ của Admin

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
admin_notification_id	BIGSERIAL	PRIMARY KEY	Khóa chính bản ghi nhận.
broadcast_id	BIGINT	FOREIGN KEY	Liên kết sang thông báo gốc notification_broadcasts.
admin_id	BIGINT	FOREIGN KEY	Admin nhận thông báo, kết nối với bảng admins.
is_read	BOOLEAN	DEFAULT FALSE	Trạng thái admin đã xác nhận đọc thông báo.
created_at	TIMESTAMP	DEFAULT NOW()	Thời điểm gửi thông báo đến hộp thư hệ thống admin.

3. Sơ đồ mối quan hệ dữ liệu (ERD)



Hình 8: Sơ đồ mối quan hệ dữ liệu ERD

4. Chiến lược lập chỉ mục tối ưu hiệu năng (Database Indexing Blueprint)

Để đảm bảo hệ thống không xảy ra tình trạng nghẽn hàng đợi kết nối (Database Connection Pool Starvation) khi có hàng ngàn yêu cầu nộp bài cùng lúc, các chỉ mục (Indexes) sau được thiết lập dựa trên tần suất truy vấn thực tế của các API.

4.1. Xác thực & Tìm kiếm tài khoản

```

-- Đẩy nhanh tốc độ kiểm tra khi Đăng nhập và hiển thị Profile
CREATE UNIQUE INDEX idx_users_username ON users(username);
  
```

```

-- Tối ưu hóa luồng Đăng ký tài khoản và khôi phục mật khẩu
CREATE UNIQUE INDEX idx_users_email ON users(email);

-- Xác thực tài khoản quản trị viên tốc độ cao
CREATE UNIQUE INDEX idx_admins_credential ON admins(username,
email);

```

4.2. Bảng xếp hạng (Leaderboard)

```

-- Lọc danh sách học viên tích cực, không cần Full Table Scan
CREATE INDEX idx_users_leaderboard ON users(is_active, deleted_
at, total_score DESC);

```

4.3. Tìm kiếm Bài tập (Problems Discovery)

```

-- Lọc danh sách bài tập theo độ khó ở màn hình trang chủ
CREATE INDEX idx_problems_filter ON problems(is_public,
difficulty);

-- Hiển thị danh sách các bài tập mới nhất
CREATE INDEX idx_problems_latest ON problems(is_public, created_
at DESC);

-- Tìm kiếm bài tập theo từ khóa không phân biệt hoa/thường
CREATE INDEX idx_problems_title_lower ON problems(LOWER(title));

```

4.4. Chấm bài & Lịch sử hệ thống (Submissions Management)

```

-- Hiển thị trang "Lịch sử nộp bài cá nhân" của học viên
CREATE INDEX idx_submissions_user ON submissions(user_id,
submitted_at DESC);

-- Hiển thị trang "Lịch sử giải bài" của một bài toán cụ thể
CREATE INDEX idx_submissions_problem ON submissions(problem_id,
submitted_at DESC);

-- Hỗ trợ Judge Engine truy vấn kết quả chi tiết theo Submission
CREATE INDEX idx_testcase_results_lookup ON testcase_
results(submission_id, testcase_id);

```

4.5. Truyền tải Thông báo (Notifications Polling)

```
-- Tối ưu API Polling đếm số thông báo chưa đọc trên thanh điều  
hướng  
CREATE INDEX idx_notifications_user_unread ON  
notifications(user_id, is_read, created_at DESC);
```

5. Ghi chú thiết kế hệ thống dữ liệu (Design Notes)

Để tối ưu hóa toàn diện cho một hệ thống Online Judge thực tế, cấu trúc cơ sở dữ liệu của ByeBug áp dụng các nguyên lý thiết kế đặc thù sau:

- **Cơ chế Xóa mềm (Soft Delete) ổn định lịch sử:** Bảng `users` sử dụng cặp trường `is_active` và `deleted_at` thay vì lệnh xóa cứng (`DELETE`). Thiết kế này giúp bảo toàn tính toàn vẹn dữ liệu (Data Integrity). Khi một tài khoản bị xóa, toàn bộ lịch sử nộp bài (`submissions`) của họ vẫn được giữ lại để làm tài nguyên thống kê hệ thống và không làm lỗi các liên kết khóa ngoại.
- **Mô hình Lưu trữ hỗn hợp tối ưu tài nguyên (Hybrid Storage Engine):** Toàn bộ dữ liệu kiểm thử nặng (`Testcases`) không lưu trực tiếp dưới dạng Binary (BLOB) trong PostgreSQL để tránh tình trạng phân mảnh và phình to dữ liệu đĩa cứng. Database chỉ lưu các Object Key độc lập; việc đọc/ghi file vật lý được nhường lại cho MinIO để đạt hiệu suất IOPS tối đa.
- **Quan hệ Nhiều-Nhiều linh hoạt cho bài tập:** Thiết kế bảng trung gian `problem_tags` kết nối giữa `problems` và `tags` giúp một bài toán có thể thuộc nhiều chủ đề thuật toán khác nhau (Ví dụ: một bài vừa thuộc tag *Array*, vừa thuộc tag *Two Pointers*), hỗ trợ bộ lọc phía Frontend hoạt động chính xác và đa dạng.
- **Tách biệt Submission và Testcase Result:** Việc tách thành hai bảng độc lập cho phép hệ thống ghi nhận chi tiết kết quả chạy (thời gian, bộ nhớ, kết quả lỗi) của **từng Testcase một** thay vì chỉ lưu kết quả tổng quan. Đây là cơ sở dữ liệu cốt lõi để hiển thị giao diện xem tiến trình chấm bài trực quan cho học viên.
- **Tách biệt Tài khoản Quản trị nội bộ:** Tách bảng `admins` độc lập hoàn toàn với bảng `users` giúp cô lập phân vùng dữ liệu, tăng cường bảo mật tầng cơ sở dữ liệu và ngăn chặn các cuộc tấn công leo thang đặc quyền (Privilege Escalation).

CHƯƠNG 3. Đặc tả hệ thống REST API ByeBug

1. Quy ước chung

Mọi tương tác truyền tải dữ liệu giữa môi trường Frontend (React) và Backend (Spring Boot) của hệ thống ByeBug được đóng gói đồng bộ thông qua giao thức **HTTP/HTTPS** với định dạng gói tin cấu trúc **JSON**.

- **Cấu hình Base URL vật lý (Localhost):** `http://localhost:8080/api`
- **Quy ước Header dành cho các API yêu cầu xác thực (Protected APIs):**

```
Authorization: Bearer <access_token>
Content-Type: application/json
```

- **Quy ước cấu trúc phản hồi lỗi hệ thống (Standard Error Response):** Để tăng tính đồng bộ và dễ dàng bóc tách thông tin tại tầng Frontend, toàn bộ lỗi xử lý nghiệp vụ hoặc lỗi hệ thống được đóng gói theo cấu trúc:

```
{
  "timestamp": "2026-06-13T12:30:00.123Z",
  "status": 400,
  "error": "Bad Request",
  "message": "Chi tiết nguyên nhân gây lỗi hệ thống.",
  "path": "/api/v1/resource-path"
}
```

2. Phân hệ API dành cho Học viên (Auth & Users APIs)

Base Path: `/api/users`

Phương thức	Đường dẫn Endpoint	Nghiệp vụ chức năng	Trạng thái bảo mật
POST	<code>/register</code>	Đăng ký tài khoản học viên mới.	Public
POST	<code>/login</code>	Đăng nhập tài khoản vào hệ thống.	Public
POST	<code>/forgot-password</code>	Yêu cầu gửi mã khôi phục mật khẩu.	Public
GET	<code>/ {username}</code>	Truy vấn chi tiết hồ sơ (Profile) người dùng.	Học viên / Admin
GET	<code>/leaderboard</code>	Truy vấn bảng xếp hạng học viên theo tổng điểm.	Public

Phương thức	Đường dẫn Endpoint	Nhiệm vụ chức năng	Trạng thái bảo mật
GET	/statistics/ {username}	Truy vấn số liệu thống kê học tập cá nhân.	Học viên / Admin
PUT	/ {username}	Cập nhật thông tin cá nhân.	Học viên
POST	/change-password/ {username}	Thực hiện đổi mật khẩu tài khoản.	Học viên
DELETE	/ {username}	Yêu cầu xóa tài khoản (Soft Delete).	Học viên / Admin

2.1. API: Đăng ký tài khoản (POST /api/users/register)

Request Body:

```
{
  "username": "student2452",
  "fullName": "Nguyễn Ngọc Thu An",
  "email": "an.nn@uit.edu.vn",
  "password": "SecurePassword123"
}
```

Response (201 Created):

```
{
  "userId": 24520063,
  "username": "student2452",
  "fullName": "Nguyễn Ngọc Thu An",
  "email": "an.nn@uit.edu.vn",
  "role": "USER",
  "totalScore": 0,
  "createdAt": "2026-06-13T12:31:00Z"
}
```

2.2. API: Đăng nhập hệ thống (POST /api/users/login)

Request Body:

```
{
  "usernameOrEmail": "student2452",
  "password": "SecurePassword123"
}
```

Response (200 OK):

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "expiresIn": 86400,
  "user": {
    "userId": 24520063,
    "username": "student2452",
    "fullName": "Nguyễn Ngọc Thu An",
    "role": "USER"
  }
}
```

3. Phân hệ API Quản trị phân quyền (Admin Auth API)

Base Path: /api/admin

Phương thức	Đường dẫn Endpoint	Nhiệm vụ chức năng	Trạng thái bảo mật
POST	/login	Đăng nhập dành riêng cho phân hệ Admin.	Public
GET	/health	Kiểm tra trạng thái hoạt động của hệ thống.	Public / Admin

4. Phân hệ API Bài tập công khai (User Problem APIs)

Base Path: /api/problems

Phương thức	Đường dẫn Endpoint	Nhiệm vụ chức năng	Trạng thái bảo mật
GET	/	Lấy danh sách bài tập hiển thị công khai (Có phân trang).	Public
GET	/ {id}	Truy vấn chi tiết đề bài, ví dụ mẫu và giới hạn.	Public
GET	/difficulty/ {difficulty}	Lọc nhanh danh sách bài tập theo cấp độ khó.	Public
GET	/search	Tìm kiếm bài tập theo từ khóa tiêu đề hoặc thẻ Tag.	Public

4.1. API: Lấy danh sách bài tập công khai (GET /api/problems)

Query Parameters: page=0, size=10

Response (200 OK):

```

{
  "content": [
    {
      "problemId": 1,
      "title": "Two Sum",
      "difficulty": "EASY",
      "timeLimitMs": 2000,
      "memoryLimitKb": 262144,
      "tags": ["Array", "Hash Table"]
    }
  ],
  "page": 0,
  "size": 10,
  "totalElements": 1,
  "totalPages": 1,
  "last": true
}

```

4.2. API: Xem chi tiết đề bài tập (GET /api/problems/{id})

Response (200 OK):

```

{
  "problemId": 1,
  "title": "Two Sum",
  "description": "Cho một mảng số nguyên `nums` và một số nguyên  
`target`...",
  "constraints": "2 <= nums.length <= 10^4",
  "difficulty": "EASY",
  "timeLimitMs": 2000,
  "memoryLimitKb": 262144,
  "allowFileSubmit": false,
  "tags": ["Array", "Hash Table"],
  "examples": [
    {
      "input": "4\n2 7 11 15\n9",
      "output": "0 1",
      "explanation": "Vì nums[0] + nums[1] == 9, ta trả về index  
0 và 1.",
      "displayOrder": 1
    }
  ]
}

```


5. Phân hệ API Lướt nộp bài (Submission APIs)

Base Path: `/api/submissions`

Phương thức	Đường dẫn Endpoint	Nhiệm vụ chức năng	Trạng thái bảo mật
POST	/	Gửi mã nguồn bài giải lên hệ thống (Nộp bài).	Học viên
GET	<code>/ {submissionId}</code>	Truy vấn kết quả chấm chi tiết của một lượt nộp.	Học viên / Admin
GET	<code>/admin</code>	Truy vấn toàn bộ danh sách lượt nộp hệ thống.	Admin
PATCH	<code>/admin/ {submissionId}/rejudge</code>	Yêu cầu Judge Engine chấm lại bài làm chỉ định.	Admin
PATCH	<code>/admin/ {submissionId}/accept</code>	Cưỡng chế công nhận bài làm đúng bằng tay.	Admin

5.1. API: Gửi mã nguồn nộp bài (POST `/api/submissions`)

Request Body:

```
{
  "problemId": 1,
  "language": "CPP",
  "sourceCode": "#include <bits/stdc++.h>\nusing namespace\nstd;\nint main() { return 0; }"
}
```

Response (202 Accepted): Ghi nhận bài nộp thành công, chờ xếp lịch chấm bất đồng bộ.

```
{
  "submissionId": 1001,
  "problemId": 1,
  "language": "CPP",
  "verdict": "PENDING",
  "score": 0,
  "submittedAt": "2026-06-13T10:00:00Z"
}
```

5.2. API: Xem kết quả chấm bài (GET `/api/submissions/{submissionId}`)

Response (200 OK):

```

{
  "submissionId": 1001,
  "verdict": "ACCEPTED",
  "score": 100,
  "totalTimeMs": 120,
  "maxMemoryKb": 2048,
  "compileError": null,
  "judgeMessage": "All testcases passed successfully.",
  "testcaseResults": [
    {
      "testcaseId": 1,
      "verdict": "ACCEPTED",
      "timeMs": 10,
      "memoryKb": 512
    }
  ]
}

```

6. Phân hệ API Quản trị bài tập dành cho Admin (Admin Problem APIs)

Base Path: /api/admin/problems

Phương thức	Đường dẫn Endpoint	Nhiệm vụ chức năng	Trạng thái bảo mật
GET	/	Lấy toàn bộ danh sách bài toán hệ thống (Gồm bài ẩn).	Admin
GET	/latest	Lấy danh sách các bài tập vừa được biên soạn mới nhất.	Admin
GET	/popular	Lấy danh sách các bài tập có lượt làm nhiều nhất.	Admin
GET	/ {problemId}	Truy vấn sâu thông tin cấu trúc nội bộ của bài toán.	Admin
POST	/	Khởi tạo một bài toán lập trình mới.	Admin
PUT	/ {problemId}	Cập nhật thông tin/đề bài/cấu hình bài toán.	Admin
POST	/ {problemId} / tests	Tải tập tin Testcase dữ liệu kiểm thử lên hệ thống.	Admin

Phương thức	Đường dẫn Endpoint	Nhiệm vụ chức năng	Trạng thái bảo mật
PATCH	/ {problemId} / visibility	Chuyển đổi trạng thái Ẩn/Hiện bài tập ra trang chủ.	Admin

6.1. API: Khởi tạo bài toán mới (POST /api/admin/problems)

Request Body:

```
{
  "title": "Two Sum",
  "description": "Given an array of integers...",
  "constraints": "1 <= n <= 10^5",
  "difficulty": "EASY",
  "timeLimitMs": 2000,
  "memoryLimitKb": 262144,
  "allowFileSubmit": false,
  "isPublic": false,
  "tags": ["Array", "Hash Table"],
  "examples": [
    {
      "input": "4\n2 7 11 15\n9",
      "output": "0 1",
      "explanation": "2 + 7 = 9",
      "displayOrder": 1
    }
  ]
}
```

7. Phân hệ API Quản lý Người dùng dành cho Admin (Admin User APIs)

Base Path: /api/admin/users

Phương thức	Đường dẫn Endpoint	Nhiệm vụ chức năng	Trạng thái bảo mật
GET	/	Truy vấn danh sách toàn bộ người dùng hệ thống.	Admin
GET	/top	Truy vấn top học viên xuất sắc phục vụ thống kê.	Admin
PATCH	/ {userId} /active	Thực hiện Khóa hoặc Mở khóa tài khoản học viên.	Admin

Phương thức	Đường dẫn Endpoint	Nhiệm vụ chức năng	Trạng thái bảo mật
DELETE	/ {userId}	Thực hiện xóa mềm (Soft Delete) tài khoản người dùng.	Admin
POST	/ {userId}/restore	Khôi phục tài khoản đã bị xóa mềm trước đó.	Admin

8. Phân hệ API Quản lý Thông báo (Notification APIs)

8.1. Phân hệ dành cho Học viên (/api/notifications)

Phương thức	Đường dẫn Endpoint	Nhiệm vụ chức năng	Trạng thái bảo mật
GET	/	Lấy danh sách thông báo được gửi đến tài khoản này.	Học viên
GET	/unread-count	Kiểm tra số lượng thông báo mới chưa đọc.	Học viên
PATCH	/ {id}/read	Đánh dấu một thông báo cụ thể là đã đọc.	Học viên

8.2. Phân hệ dành cho Admin (/api/admin/notifications)

Phương thức	Đường dẫn Endpoint	Nhiệm vụ chức năng	Trạng thái bảo mật
POST	/	Phát hành một thông báo Broadcast đến toàn hệ thống.	Admin
GET	/	Xem lịch sử các thông báo diện rộng đã phát hành.	Admin
GET	/inbox	Xem hộp thư thông báo hệ thống nội bộ của Admin.	Admin
PATCH	/inbox/ {id}/read	Đánh dấu thông báo nội bộ của admin đã đọc.	Admin
GET	/inbox/unread-count	Đếm số thông báo quản trị hệ thống chưa xác nhận.	Admin

9. Phân hệ API Thống kê & Dashboard (Home / Overview APIs)

Phương thức	Đường dẫn Endpoint	Nhiệm vụ chức năng	Trạng thái bảo mật
GET	/api/home/summary	Lấy tổng quan thông tin trang chủ của Học viên.	Học viên
GET	/api/admin/overview	Lấy các chỉ số biểu đồ Dashboard tổng quan Admin.	Admin

10. Định nghĩa hệ thống mã trạng thái (Standard HTTP Status Codes)

Hệ thống cam kết sử dụng các mã trạng thái tiêu chuẩn của giao thức HTTP để phản hồi kết quả:

Mã	Ý nghĩa
200 OK	Các yêu cầu đọc dữ liệu (GET), cập nhật (PUT/PATCH) hoặc xử lý nghiệp vụ thành công.
201 Created	Khởi tạo thành công các đối tượng thực thể mới (Users, Problems).
202 Accepted	Tiếp nhận gói tin thành công và xếp lịch xử lý ngầm (Áp dụng cho luồng nộp bài Submissions).
400 Bad Request	Cấu trúc gói tin JSON gửi lên bị sai cú pháp, thiếu trường bắt buộc hoặc sai định dạng dữ liệu.
401 Unauthorized	Token đính kèm bị hết hạn, không hợp lệ hoặc người dùng chưa thực hiện đăng nhập.
403 Forbidden	Tài khoản học viên cố tình gọi các API thuộc đặc quyền Quản trị viên (/api/admin/).
404 Not Found	Tài nguyên yêu cầu (Bài tập, Lướt nộp bài, Tài khoản) không tồn tại trên hệ thống.
409 Conflict	Xảy ra xung đột tài nguyên dữ liệu hệ thống (Trùng lặp tên username hoặc địa chỉ email).
500 Internal Server Error	Lỗi phát sinh ngoài dự kiến xuất phát từ mã nguồn máy chủ.

CHƯƠNG 4. Kiến trúc thành phần và triển khai hệ thống ByeBug

1. Môi trường công nghệ nền tảng (Technology Stack)

Hệ thống ByeBug được đóng gói và vận hành dựa trên sự phối hợp của các công nghệ hiện đại, đảm bảo tính bất đồng bộ và khả năng chịu tải tốt:

Tầng chức năng	Công nghệ sử dụng	Vai trò chi tiết
Presentation (Frontend)	React 19, TypeScript, Vite, React Router, Axios, Monaco Editor, Recharts	Xây dựng giao diện Single Page Application (SPA), tích hợp trình soạn thảo code thông minh và hệ thống biểu đồ trực quan.
Business Logic (Backend)	Java Spring Boot, Spring MVC, Spring Security, JWT, Spring Data JPA, Hibernate	Xử lý toàn bộ logic nghiệp vụ hệ thống, quản lý phiên xác thực, phân quyền endpoint.
Execution (Judge Engine)	Java Spring Boot Worker, Redis Consumer, Docker Java API	Khởi xử lý ngắt tiêu thụ tác vụ chấm bài, tương tác với hạ tầng ảo hóa để chạy mã nguồn người dùng.
Relational Database	PostgreSQL 17	Lưu trữ dữ liệu quan hệ có tính toàn vẹn cao (Users, Problems, Submissions).
Cache & Message Broker	Redis 7	Tầng đệm tối ưu truy vấn dữ liệu và đóng vai trò hàng đợi phân phối tác vụ (Job Queue).
Object Storage	MinIO (S3 Compatible)	Kho lưu trữ tệp tin Testcase cô lập, giải tải dung lượng lưu trữ cho Database chính.
Virtualization	Docker, Docker Compose	Đóng gói môi trường, cô lập tài nguyên phần cứng và điều phối các dịch vụ.

2. Cấu trúc tổ chức mã nguồn (Source Code Directory Tree)

Mã nguồn dự án ByeBug được tổ chức theo kiến trúc Monorepo giúp quản lý tập trung và đồng bộ hóa biến môi trường cấu hình một cách dễ dàng:

```
ByeBug-main/  
├── frontend/                # Ứng dụng giao diện người dùng UI  
    (React + TypeScript)  
├── backend/                 # Ứng dụng cung cấp các dịch vụ  
    RESTful API (Spring Boot)
```

```

└─ judge-engine/           # Khối xử lý (Worker) chấm bài tự
động bất đồng bộ
└─ docker-compose.yml      # Cấu hình khởi chạy hạ tầng đồng bộ
tại môi trường local/dev
└─ docker-compose.prod.yml # Cấu hình gợi ý tối ưu tài nguyên
mạng khi triển khai production
└─ init.sql                # Tập lệnh cấu trúc tạo bảng và
thiết lập dữ liệu mẫu ban đầu
└─ docs/                  # Kho tài liệu hướng dẫn vận hành và
đặc tả sẵn có

```

3. Phân rã cấu trúc Phân tầng lớp của API Backend

Hệ thống Backend được tổ chức chặt chẽ theo mô hình kiến trúc nhiều lớp (Multi-layered Architecture) giúp tăng tính tái sử dụng code và dễ bảo trì:

```

Controller (Tiếp nhận HTTP request, xác thực JWT, phân trang dữ
liệu)
↓
Service (Xử lý logic nghiệp vụ, quản lý Transactions, điều phối
dữ liệu)
↓
Repository (Thực hiện ánh xạ thực thể ORM, thực thi truy vấn
PostgreSQL)
↓
Data Infrastructure (PostgreSQL / Redis / MinIO Storage)

```

3.1. Tầng Controller — Điều hướng API

Tên Controller	Nghịệp vụ xử lý
UserController	Xử lý đăng ký, đăng nhập, hồ sơ cá nhân, bảng xếp hạng Leaderboard và số liệu thống kê.
ProblemController	Cung cấp danh sách và thông tin chi tiết các bài toán hiển thị công khai.
SubmissionController	Tiếp nhận mã nguồn nộp bài, xem lịch sử và điều phối lệnh Rejudge/Accept của admin.
AdminController	Xử lý xác thực đăng nhập phân hệ quản trị và kiểm tra sức khỏe API (Health check).
AdminProblemController	Phục vụ biên soạn bài tập mới, cấu hình ràng buộc và đăng tải dữ liệu Testcase.

Tên Controller	Nhiệm vụ xử lý
<code>AdminController</code>	Giám sát trạng thái hoạt động, khóa/mở khóa hoặc xóa mềm tài khoản học viên.
<code>NotificationController</code>	Đẩy thông báo cá nhân và cập nhật trạng thái đã đọc cho học viên.
<code>AdminNotificationController</code>	Quản lý và phát hành các thông báo diện rộng (Broadcast) hệ thống.
<code>AdminOverviewController</code>	Cung cấp các chỉ số tổng hợp tổng quan cho trang đồ thị Dashboard admin.
<code>HomeController</code>	Tổng hợp thông tin tóm tắt hiển thị tại trang chủ cá nhân của học viên.

3.2. Tầng Service — Xử lý Nghiệp vụ

- **UserService:** Đảm nhiệm việc băm mật khẩu (BCrypt), xác thực thông tin, tính toán điểm tích lũy.
- **ProblemService & AdminProblemService:** Quản lý logic đồng bộ cấu trúc bài toán, liên kết thẻ dữ liệu (Tags).
- **SubmissionService:** Tiếp nhận bài giải, sinh bản ghi trạng thái PENDING, gọi lệnh chấm lại từ Admin.
- **JudgeQueueProducer:** Đóng gói thông tin lượt nộp bài thành JudgeJob và đẩy vào cấu hình Redis Queue.
- **TestcaseUploadService:** Thực hiện đẩy tệp tin .in/.out lên MinIO và ghi nhận Metadata xuống PostgreSQL.
- **NotificationService:** Xử lý luồng phân phối thông báo diện rộng bất đồng bộ.
- **JwtService:** Đóng gói thông tin định danh, thiết lập thời gian hết hạn và giải mã chữ ký điện tử Token JWT.

3.3. Tầng Repository — Truy cập dữ liệu

Sử dụng Spring Data JPA kế thừa từ Hibernate để thực thi các câu lệnh truy vấn tối ưu xuống PostgreSQL:

- Tìm kiếm người dùng qua chỉ mục `username` hoặc `email`.
- Quét và trích xuất danh sách Leaderboard sắp xếp theo `total_score DESC`.
- Thực hiện các câu lệnh tìm kiếm tương đối (Like Query) kết hợp chuẩn hóa viết thường `LOWER(title)`.
- Đếm số lượng các thông báo chưa đọc (`is_read = false`) để tối ưu hóa tần suất Polling từ Frontend.

4. Phân rã cấu trúc Phân hệ Frontend (React Application)

Ứng dụng Frontend được thiết kế theo kiến trúc Module hóa (Component-Based Architecture) giúp tối ưu hóa luồng render của Virtual DOM.

4.1. Tầng Giao tiếp API (`frontend/src/api`)

Tên File cấu hình	Vai trò chức năng
<code>axios.ts</code>	Khởi tạo Axios Instance, thiết lập <code>baseURL</code> và cấu hình Interceptors để tự động đính kèm Token JWT vào Header của mọi request.
<code>userApi.ts</code>	Giao tiếp các endpoint liên quan đến tài khoản, phiên đăng nhập, xếp hạng và profile.
<code>problemApi.ts</code>	Gọi API lấy danh sách bài tập công khai, thực hiện tìm kiếm, phân trang và đọc đề bài.
<code>submissionApi.ts</code>	Gửi mã nguồn lên server và thực hiện Polling liên tục để lấy kết quả chấm từ Worker.
<code>adminApi.ts</code>	Tổng hợp toàn bộ các lệnh gọi đặc quyền dành riêng cho giao diện quản trị Admin.
<code>homeApi.ts / notificationApi.ts</code>	Xử lý luồng lấy dữ liệu tóm tắt trang chủ và danh sách hộp thư thông báo.

4.2. Hệ thống Trang (Main Pages) & Thành phần (Components) nổi bật

- **Pages:** Tách biệt rõ ràng giữa khu vực công khai (Login, Register, Problems) và khu vực nội bộ (Profile, Submission History) cùng phân hệ trang quản trị độc lập (Admin Dashboard Pages).
- **CodeEditor Component:** Tích hợp bộ thư viện Monaco Editor (nền tảng của VS Code) mang lại trải nghiệm gõ code chuyên nghiệp cho học viên, hỗ trợ highlight cú pháp theo từng ngôn ngữ lựa chọn.
- **ResizableProblemLayout:** Sử dụng kỹ thuật Layout linh hoạt cho phép học viên kéo dẫn tùy biến kích thước giữa phân vùng đọc đề bài (bên trái) và trình soạn thảo code (bên phải).
- **ProblemStatementPanel:** Sử dụng bộ parser Markdown tích hợp hỗ trợ hiển thị các công thức toán học/thuật toán phức tạp trực quan.

5. Kiến trúc Chuyên biệt của khối chấm bài Judge Engine

Khối Judge Engine được xây dựng độc lập hoàn toàn dưới dạng một dịch vụ chạy ngầm (Background Worker), giúp bảo vệ API chính không bị ảnh hưởng hiệu năng khi có lượng bài nộp lớn cần biên dịch nặng.

5.1. Quy trình vận hành cốt lõi (Core Workflow)

1. Lắng nghe và tiêu thụ (Consume) thông điệp `JudgeJob` xuất hiện từ hàng đợi Redis Queue.
2. Truy vấn chi tiết thông tin bài làm và metadata tương ứng từ cơ sở dữ liệu PostgreSQL.
3. Tải tệp tin kiểm thử dữ liệu thực tế (`.in / .out`) từ bộ lưu trữ MinIO về vùng nhớ đệm tạm thời.
4. Tương tác với Docker API để khởi tạo một Sandbox Container cô lập hoàn toàn tài nguyên, thực hiện biên dịch và chạy code của người dùng.
5. Thực hiện so sánh dữ liệu đầu ra và ghi nhận chi tiết kết quả từng bộ test về PostgreSQL, cập nhật verdict tổng cho lượt nộp bài.

5.2. Tổ chức Modules ứng dụng áp dụng Strategy Pattern

Để hệ thống có khả năng mở rộng thêm nhiều ngôn ngữ mới một cách dễ dàng, Judge Engine áp dụng thiết kế Mẫu chiến lược (Strategy Pattern):

Thành phần lớp	Vai trò chức năng
<code>JudgeWorker</code>	Lắng nghe, nhận diện gói tin từ hàng đợi Redis Consumer.
<code>JudgeService</code>	Điều phối tổng thể luồng chạy của một bài toán (Tải file, gọi bộ chấm, cập nhật DB).
<code>DockerService</code>	Tương tác trực tiếp với Docker Daemon qua Socket, quản lý vòng đời Container Sandbox.
<code>TestcaseStorageService</code>	Kết nối, tải dữ liệu testcase bảo mật từ các Private Bucket trên MinIO.
<code>JudgeStrategy</code>	Giao diện mẫu (Interface) định nghĩa các phương thức biên dịch, chạy và thu thập tài nguyên hệ thống chung.
<code>CppJudgeStrategy</code>	Hiện thực hóa chiến lược chấm cụ thể cho ngôn ngữ C++ (Gọi trình biên dịch <code>g++</code> , thiết lập cờ tối ưu).

6. Điều phối Hạ tầng ảo hóa Docker Compose

Hệ thống cung cấp file `docker-compose.yml` liên kết toàn bộ 7 dịch vụ chính chạy trên một mạng ảo nội bộ cô lập (`byebug-network`).

Giải trình Kỹ thuật quan trọng — Docker-in-Docker Socket Binding: Để dịch vụ `judge-engine` chạy bên trong container có thể ra lệnh khởi chạy các Sandbox Container (`oj-cpp-runner`) ra môi trường máy chủ vật lý, hệ thống sử dụng kỹ thuật gắn kết Volume Socket:

```
volumes:  
  - /var/run/docker.sock:/var/run/docker.sock
```

Thiết kế này giúp Worker giao tiếp trực tiếp với Docker Daemon của máy Host, đảm bảo hiệu năng khởi tạo Container con đạt tốc độ tối đa dưới 50ms mà vẫn giữ được tính cô lập an toàn cho Sandbox.

7. Hướng dẫn vận hành khởi chạy tại môi trường nội bộ (Local Deployment Guide)

7.1. Cấu hình biến môi trường hệ thống

Tạo lập tệp tin `.env` từ file mẫu cấu hình để định nghĩa toàn bộ hệ thống cổng (Ports) và thông tin xác thực mật khẩu an toàn:

```
cp .env.example .env
```

Cấu hình các giá trị kết nối cốt lõi tương ứng như: `VITE_PORT=5173`, `SPRING_PORT=8080`, `MINIO_CONSOLE_PORT=9001`, `VITE_API_BASE_URL=http://localhost:8080`.

7.2. Lệnh khởi chạy đồng bộ

Khởi chạy tiến trình build mã nguồn và dựng toàn bộ hạ tầng ngậm chỉ bằng một câu lệnh duy nhất:

```
docker compose up --build -d
```

Hệ thống tự động đọc file `init.sql` để thiết lập cấu trúc các bảng và nạp dữ liệu môi trường ban đầu. Người vận hành có thể giám sát sức khỏe của hệ thống thông qua các cổng dịch vụ Endpoint `/actuator/health` đã được cấu hình sẵn.

8. Hệ thống Kiểm thử tự động tích hợp (Integrated Unit Tests)

Để đảm bảo tính ổn định và chính xác của các thuật toán xử lý logic lỗi trước khi deploy, dự án tích hợp sẵn bộ kiểm thử tự động (Unit Tests) bao phủ các dịch vụ cốt lõi:

- **Học viên & Xác thực:** `UserServiceTest`, `JwtServiceTest`.
- **Quản trị hệ thống:** `AdminProblemServiceTest`, `AdminUserServiceTest`, `TestcaseUploadServiceTest`.
- **Luồng chấm bài cốt lõi:** `CppJudgeStrategyTest` (Kiểm thử độ chính xác của bộ chấm C++, đo lường sai số thời gian và bộ nhớ).

Lệnh thực thi kiểm thử toàn diện cho phân hệ Backend:

```
cd backend && ./mvnw test
```

Lệnh thực thi kiểm thử toàn diện cho phân hệ Judge Engine:

```
cd judge-engine && ./mvnw test
```

CHƯƠNG 5. Đánh giá hiệu năng hệ thống ByeBug

1. Mục tiêu đánh giá và Phạm vi đo lường

Phân đánh giá hiệu năng nhằm chứng minh hệ thống ByeBug có thể xử lý các tác vụ nghiệp vụ chính với độ trễ hợp lý, thông lượng cao và tỷ lệ lỗi thấp dưới áp lực truy cập đồng thời. Phạm vi đo lường tập trung vào hai phân vùng:

- Hiệu năng phản hồi HTTP tại tầng REST API (Đăng nhập, đọc đề bài, tạo lượt nộp).
- Năng lực điều phối, biên dịch và cô lập tài nguyên bất đồng bộ tại tầng Judge Engine Worker.

2. Công cụ và Môi trường đo kiểm (Testbed Setup)

2.1. Hệ thống công cụ sử dụng

- **k6 (Grafana):** Sử dụng tập lệnh JavaScript để giả lập các kịch bản người dùng ảo (Virtual Users — VUs) đồng thời truy cập hệ thống.
- **Docker Stats CLI & Spring Boot Actuator:** Giám sát mức độ chiếm dụng CPU, RAM vật lý của các container trong suốt quá trình test tải.

2.2. Cấu hình môi trường thử nghiệm

Toàn bộ các kịch bản kiểm thử được thực thi trực tiếp trên hạ tầng giả lập local (Môi trường phát triển của nhóm):

- **Host Machine:** CPU Intel Core i7-11800H @ 2.30GHz (8 Cores, 16 Threads) | RAM 16GB DDR4 | OS: Windows 11 Pro + WSL2 (Ubuntu 22.04 LTS).
- **Docker Compose Allocations:**
 - `backend-api`: Giới hạn tối đa 2GB RAM.
 - `judge-engine`: Cho phép sử dụng tối đa tài nguyên Host để khởi tạo container sandbox.
 - `postgres-db` & `redis-queue`: Cấu hình container mặc định trên nền Alpine.

3. Kết quả Kiểm thử tải hệ thống các Phân hệ API (Load Testing Results)

Nhóm tiến hành kích hoạt các kịch bản test k6 với thời gian Ramp-up 30 giây, duy trì tải đỉnh trong vòng 3 phút và Ramp-down 30 giây để thu thập các chỉ số: Độ trễ trung bình (Avg), Độ trễ phân vị 95% (p_{95}) và Thông lượng (Throughput).

3.1. Bảng tổng hợp hiệu năng tầng REST API

Dưới đây là số liệu thực tế ghi nhận được từ hệ thống đo lường của k6:

Kịch bản nghiệp vụ	Số User ảo (VUs)	Độ trễ trung bình (Avg)	Phân vị tối hạn (p_{95})	Thông lượng (Throughput)	Tỷ lệ lỗi gói tin (Error %)	Đánh giá đạt chuẩn
Đăng nhập (POST /login)	10	45 ms	92 ms	4.8 req/s	0.00%	Đạt
Đăng nhập (POST /login)	50	124 ms	285 ms	22.4 req/s	0.00%	Đạt (CPU tăng do hash BCrypt).
Tải danh sách bài (GET /)	100	18 ms	42 ms	85.1 req/s	0.00%	Xuất sắc (Tối ưu nhờ Pagination/Index).
Xem chi tiết bài (GET /1)	100	12 ms	31 ms	92.4 req/s	0.00%	Xuất sắc (Dữ liệu tĩnh, tải nhanh).
Gửi bài giải (POST /)	30	38 ms	88 ms	14.2 req/s	0.00%	Đạt (Backend xử lý bất đồng bộ).

4. Kết quả Đánh giá Năng lực và Tốc độ Chấm bài (Judge Latency Results)

Để đo lường hiệu năng của tầng Judge Engine Worker, nhóm thiết lập bài toán kiểm thử gồm 3 bộ testcases ẩn. Thực hiện bắn luồng nộp bài đồng thời (Batch Submissions) mã nguồn C++ chính xác (Verdict: ACCEPTED) và thay đổi số lượng Worker Instances để chứng minh khả năng mở rộng hàng ngang (Horizontal Scaling).

4.1. Bảng đo lường thời gian hoàn tất chấm bài (Turnaround Time)

Tổng số bài nộp đồng thời	Số Judge Worker hoạt động	Số testcase / bài	Thời gian chấm trung bình (Avg)	Phân vị tối hạn (p_{95})	Trạng thái hàng đợi Redis	Ghi chú vận hành
10 Submissions	1 Worker	3 Tests	1.12 s	1.45 s	Trống (Xử lý cuộn chiều liên)	Hệ thống vận hành mượt mà.

Tổng số bài nộp đồng thời	Số Judge Worker hoạt động	Số testcase / bài	Thời gian chấm trung bình (Avg)	Phân vị tối hạn (p_{95})	Trạng thái hàng đợi Redis	Ghi chú vận hành
30 Submissions	1 Worker	3 Tests	4.85 s	8.21 s	Tích lũy đỉnh 22 Jobs	Xuất hiện độ trễ do nghẽn worker đơn.
30 Submissions	2 Workers	3 Tests	2.31 s	3.94 s	Tích lũy đỉnh 8 Jobs	Scale hiệu quả: Thời gian giảm ~50%.
50 Submissions	3 Workers	3 Tests	2.18 s	3.65 s	Giải tỏa queue dưới 4s	Hệ thống chịu tải đỉnh rất tốt.

5. Phân tích Kỳ vọng và Định hướng Tối ưu hệ thống (Performance Bottleneck Analysis)

Dựa trên các chỉ số thực chứng thu được từ hệ thống, nhóm rút ra các phân tích kỹ thuật quan trọng:

1. Hiệu năng Tầng API: Nhờ áp dụng cơ chế phân trang (Pagination) nghiêm ngặt kết hợp với Database Indexing trên các trường tra cứu chính (`is_public`, `difficulty`, `user_id`), độ trễ phân vị p_{95} của các API đọc danh sách luôn duy trì ở mức dưới 50ms ngay cả khi có 100 user ảo quét liên tục.

2. Độ trễ phản hồi luồng nộp bài (Submit API vs Judge Latency): Đúng như thiết kế kiến trúc hướng sự kiện, API `POST /api/submissions` phản hồi cực nhanh (~38ms) vì Spring Boot chỉ làm nhiệm vụ ghi nhận metadata vào Postgres và đẩy bản tin vào Redis Queue. Học viên không bị treo trình duyệt để đợi kết quả.

3. Điểm nghẽn tại Judge Worker (Docker Startup Overhead): Khi 1 Worker phải xử lý cùng lúc 30 bài nộp, thời gian chấm bị kéo dài lên hơn 4 giây. Qua theo dõi bằng `docker stats`, điểm nghẽn không nằm ở dung lượng RAM mà nằm ở overhead tài nguyên CPU khi sinh (fork) liên tiếp các Docker Container cô lập cho mỗi lượt chạy. Khi nhóm nâng cấu hình lên 3 Workers chạy song song, thời gian chấm lập tức được kéo giảm xuống mức lý tưởng (~2.18s), chứng minh kiến trúc bất đồng bộ qua Redis Queue của ByeBug có khả năng mở rộng (Scale-out) cực kỳ tuyến tính và hiệu quả.

6. Kết luận Chương Hiệu năng

Các số liệu thực chứng từ k6 và hệ thống giám sát hạ tầng đã chứng minh mô hình kiến trúc phân tách của ByeBug đạt hiệu năng tối ưu. Hệ thống hoàn toàn đáp ứng được nhu cầu vận hành thực tế tại các phòng Lab trường đại học với quy mô một buổi thi tập trung từ 50–100 sinh viên nộp bài liên tục mà không gây ra tình trạng nghẽn mạch hay sập cục bộ bất kỳ phân hệ nào.